

Tyler Vander Mooren

CYB 310: Malware Analysis

Professor Troy Adams

February 14, 2026

Lab 2 – Static Analysis & VMs

Document-based malware remains one of the most common initial access vectors in modern cyberattacks. Attackers frequently embed malicious macros and staged scripts within Office and PDF documents. The samples analyzed in this lab demonstrate three distinct document-based threat models, macro-enabled malware loaders, phishing-based documents and potentially dormant or environment dependent malicious files. The analysis was conducted using the App.Any.Run platform to observe process trees, command-line activity, file activity and system interactions. The following samples were analyzed:

- mal_office_section4.zip
- mal_office_section9.zip
- mal_pdf.zip
- mal_pdf_2.zip

Analysis of Macro-Based Microsoft Office Documents

Both Office zips contained a malicious Word document that exhibited nearly identical behavior and both functioned as macro-based malware loaders. Upon opening the malicious Word documents, WINWORD.EXE spawns an abnormal sequence of child processes. WINWORD.EXE->cmd.exe->cscript.exe->powershell.exe. This staged execution strongly indicates that malicious macro activity is happening. Regular legitimate Word Documents do not spawn command interpreters or scripting engines. Upon examining the PowerShell execution, it

uses -ExecutionPolicy bypass and -nopprofile to override PowerShell's security restrictions. This allows the scripts to execute regardless of the configured policies. While the -nopprofile is there for further protection against defensive scripts and monitoring configurations, this shows deliberate evasion. The samples also dropped multiple files with naming conventions like abodeacd-update.bat trying to mimic legitimate Adobe update components, showcasing masquerading techniques. Use of a batch file followed by VBScript and then the PowerShell execution shows a structured, multi-stage loader design rather than a single embedded payload. Further system reconnaissance was done through Windows Management Instrumentation (WMI), the Office documents specifically queried Win32_OperatingSystem. Some additional information that was collected was the operating system enumeration, proxy configuration checks, language setting verification and retrieval of script execution paths. These actions suggest that environment configuration is collected prior to the payload delivery; it does this to ensure compatibility and maximize impact. Further evasion techniques were utilized including disabling PowerShell trace logs and introducing execution delays using a ping command. Both samples attempted to download a remote payload via PowerShell but resulted in the connection failing in the sandbox environment. The combination of staged macro execution, reconnaissance, execution policy bypass and remote payload attempts clearly demonstrates malicious loader behavior. These characteristics align with real world phishing attachment campaigns that use Office documents as the initial loaders for the secondary payloads.

Analysis of PDF-Based Documents

In contrast to the Office samples, the PDF documents did not spawn command shells or scripting during execution. The first sample mal_pdf was classified as phishing and when opened in Adobe Acrobat it launched multiple AcroCEF.exe processes but these processes are normal

components of Acrobat's rendering. Also, there were no abnormal child processes with no evidence of PowerShell execution, shell spawning, execution policy modification or remote payload retrieval. Because of the lack of exploit style behavior occurring the malicious classification likely stems from the document content rather than executable scripting. Phishing PDFs will often contain embedded hyperlinks, impersonation content or instructions designed to deceive users into revealing credentials. However, there is no information present in the PDF when it is opened in the sandbox but it does receive the Windows Registry Key and security settings of Internet Explorer through the AcroCEF.exe processes, this might be an attempt to get more information about the environment. The sample represents a malware model more focused on social engineering rather than system-level exploitation. The second PDF sample also did not display observable malicious behavior within the sandbox. It opened normally and spawned standard rendered, GPU and utility processes. There were no suspicious command-line parameters or child processes being detected. Despite having no runtime exploit behavior the document was still flagged as malicious. This suggests that both PDF documents may require some user interaction, be dependent on the Adobe version used, or rely on environmental conditions not present in the sandbox. The second sample does highlight an important limitation of dynamic analysis, that the absence of visible malicious behavior does not confirm a file is benign.

Threat Comparison

The analyzed documents fall into three primary categories:

1. Macro-based malware loaders (Office samples)
2. Content-based phishing documents (PDF samples)
3. Dormant or environment-dependent documents (PDF samples)

The Office samples clearly executed staged scripts and attempted remote payload retrieval, demonstrating active malware delivery. In contrast, the PDF samples relied either on social engineering or conditional execution rather than overt command spawning. Overall, the samples reinforce the importance of evaluating execution chains, scripting behavior, reconnaissance activity and evasion techniques when analyzing document-based threats. It also demonstrates some of the limitations of sandbox-based analysis, especially with `app.any.run` being time restricted and the continued need for structural static inspection when dealing with potentially malicious documents.